

ODMA–URA research record

The factorised explicit framework

General codebook algebra, implicit operators, and learned D0/D1

Stage	3 of 4
Updated	30 August 2026
Status	Small- B framework validated; jobs 021–022 complete

This report is a chronological research record. Measured results, interpretations, and open hypotheses are labelled separately. Detailed numerical ledgers remain in **results/**.

1 Purpose

The explicit experiments used a different construction routine for each codebook. The framework replaced those special cases by one factorisation, allowing representation, encoder learning, and decoder learning to be compared without changing the outer experiment contract. It also clarified two different meanings of “implicit”: avoiding a stored $n \times M$ matrix is useful, but it does not remove an $M = 2^B$ decoder state.

2 Full global construction and shapes

Let $M = 2^B$ and let L be the number of additive components. Component ℓ has:

$$R_{\ell,q} \in \mathbb{C}^{n \times d_\ell}, \quad q = 0, \dots, Q_\ell - 1, \quad C_\ell \in \mathbb{C}^{d_\ell \times V_\ell}.$$

Define the complete product bank

$$B_\ell = [R_{\ell,0}C_\ell \mid \dots \mid R_{\ell,Q_\ell-1}C_\ell] \in \mathbb{C}^{n \times (Q_\ell V_\ell)}.$$

A sparse selector $U_\ell \in \{0, 1\}^{Q_\ell V_\ell \times N_\ell}$ retains N_ℓ legal local atoms. A lookup $T_\ell \in \{0, 1\}^{N_\ell \times M}$ has one nonzero per column and maps every global message to one local atom. The global codebook is

$$\Phi = \sum_{\ell=1}^L B_\ell U_\ell T_\ell \in \mathbb{C}^{n \times M}.$$

For global counts $a \in \mathbb{R}^M$, the noiseless signal is

$$y = \Phi a = \sum_{\ell} B_\ell U_\ell s_\ell, \quad s_\ell = T_\ell a \in \mathbb{R}^{N_\ell}.$$

s_ℓ aggregates every global message that uses the same local atom. This is the core many-to-one compression, but the explicit backend still needs the length- M mapping T_ℓ and a length- M output to distinguish messages.

2.1 Column-level interpretation

If message m maps to local atom $i_\ell(m)$, and that atom represents pair $(q_\ell(m), v_\ell(m))$, then

$$\phi_m = \sum_{\ell=1}^L R_{\ell,q_\ell(m)} c_{\ell,v_\ell(m)}.$$

This can represent a dense global codebook, ODMA placements, shared product alphabets, restricted valid pairs, multiple section contributions, and fixed or learned local matrices. Representability is not the same as compact scalability: an arbitrary T_ℓ contains M choices.

3 How implementation avoids materialising the global matrix

The implemented component stores integer arrays describing the N_ℓ legal atoms and a length- M msg_to_atom lookup. The forward path performs

$$a \xrightarrow{T_\ell} s_\ell \xrightarrow{U_\ell} \text{coefficients on } (q, v) \xrightarrow{C_\ell} \text{one local vector per } q \xrightarrow{R_{\ell,q}} y_\ell,$$

then sums $y = \sum_{\ell} y_{\ell}$. The adjoint reverses these operations:

$$r \xrightarrow{R_{\ell,q}^H} r_{\ell,q} \xrightarrow{C_{\ell}^H} \text{scores on } (q, v) \xrightarrow{U_{\ell}^H} \text{local atom scores} \xrightarrow{T_{\ell}^H} \Phi_{\ell}^H r.$$

Thus normal training uses memory proportional to factors plus M , rather than nM . Selected columns or the entire matrix can still be materialised for small-system diagnostics.

3.1 Gradient flow

For loss $\mathcal{L}(y)$ and upstream vector $g_y = \partial\mathcal{L}/\partial y$, automatic differentiation follows the same adjoints. For example, with $Z_{\ell,q} = C_{\ell} X_{\ell,q}$ and $y_{\ell} = \sum_q R_{\ell,q} Z_{\ell,q}$,

$$\frac{\partial\mathcal{L}}{\partial Z_{\ell,q}} = R_{\ell,q}^H g_y, \quad \frac{\partial\mathcal{L}}{\partial C_{\ell}} = \sum_q (R_{\ell,q}^H g_y) X_{\ell,q}^H.$$

The gradient with respect to counts is $\partial\mathcal{L}/\partial a = \Phi^H g_y$. Fixed integer selectors U, T receive no gradient; learning them requires a discrete search or a separate relaxation. Learning C or continuous R does not.

4 D0: exact data consistency, learned scalar denoising

After matched channel combining, initialise $a^0 = 0 \in \mathbb{R}^{P \times M}$ for batch size P . At layer t ,

$$r^t = y - \Phi a^t, \quad g^t = \Re\{\Phi^H r^t\}, \quad u^t = a^t + \eta_t g^t,$$

where $\eta_t > 0$ is scaled by an implicit estimate of $\|\Phi\|_2^2$. With known K_a , D0 uses $\rho = K_a/M$ and produces calibrated support logits

$$\lambda_j^t = \text{clip} \left[\gamma_t \frac{u_j^t - 1/2}{\tau_t} + \beta_t \log \frac{\rho}{1 - \rho} + b_t, -30, 30 \right].$$

The soft proposal is $\tilde{a}^t \propto \sigma(\lambda^t)$, normalised so $\mathbf{1}^T \tilde{a}^t = K_a$, and damping forms a^{t+1} . Only the final evaluation output is projected to nonnegative integer counts.

The exact Φ/Φ^H pair preserves physical data consistency. Learned parameters calibrate evidence, prior strength, temperature, step size, and damping; there is no learned dense message-to-message matrix. Gradients flow through every unrolled layer to the decoder scalars and any learnable encoder matrices.

5 D1: nonlocal factor context

D1 starts with the identical D0 map and adds a learned proximal correction. For a single product component it labels each message by its pattern q_j and local value v_j . Per-message features include u_j^t, a_j^t, g_j^t , base logit, pattern mass, residual scale, noise scale, and prior logit. Slot attention pools context:

- within messages sharing q ;
- across patterns sharing v ;
- globally across all messages.

A small correction network adds Δ_j^t to the D0 logit. It is zero-initialised, so before training D1 is exactly D0. This construction is linear rather than quadratic in M for fixed slot count, but it remains M -indexed and is several times more expensive than D0.

6 Validation before performance claims

The framework passed exact or numerical checks for implicit versus materialised forward maps, adjoint identities, selected-column construction, legacy ODMA representation, reduced sectioned CCS representation, and tiny exact count ML. The normal $B = 14$, $M = 16384$ training path completed without an $n \times M$ matrix. These checks establish that the factor operations implement the intended small- B algebra; they do not validate an arbitrary decoder or establish paper-faithful CCS-AMP equivalence.

7 Jobs 021–022: what was learned

The controlled bank used $L = 1$, $U = T = I$, and $d = n$. It compared dense, shared product, independently sparse-global, and four-mask ODMA encoders at four (B, n) geometries: (12, 128), (12, 256), (14, 128), and (14, 256).

High-SNR mean PUPE	12, 128	12, 256	14, 128	14, 256
Dense D1	.2912	.2185	.3556	.3099
Product D1 average	.3118	.2268	.3717	.3264
Sparse-global D1	.3399	.2182	.3714	.3040
ODMA D1	.5417	.4561	.5886	.5295

Table 1: Average over four loads at 8 and 12 dB. Cells contain limited evaluation samples; differences of a few PUPE points are not reliable rankings.

Across all evaluation cells, D0 improved over matched filtering by about 0.004 PUPE on average. D1’s improvement was about 0.042 at 8/12 dB, but D1 also helped dense codebooks; the experiment therefore supports nonlocal learned context, not a uniquely product-factor gain. Product sharing was competitive but slightly worse than dense on average, and learning C was mixed across geometries. Median D1 runtime was 5.2 times D0.

The most consequential result was morphological. Sparse-global and ODMA both used 25% nonzeros per column, but sparse-global gave each message an independently sampled support. ODMA reused only four masks. Sparse-global row-energy coefficient of variation was 0.023–0.053; ODMA’s was 0.839–0.875, with about 30% of rows unused. Sparse-global was within 0.016 PUPE of dense in every geometry and essentially tied at $n = 256$.

Evidence status: Measured finite experiment. It shows that this ODMA support reuse is harmful under the tested decoder; it does not show that sparsity is universally harmless or that sparse-global storage is scalable.

8 Scalability boundary

Even without a stored Φ , the explicit framework keeps T_ℓ and a^t indexed by all M messages. Going from $B = 14$ to $B = 100$ multiplies that dimension from 16,384 to approximately 1.27×10^{30} . Factorised matrix arithmetic alone cannot cross this boundary. The next stage must generate a message’s local choices procedurally and decode from local states without ever expanding them back to every global message.